

TD7: Ingeniería de Datos

Procesamiento de datos

Introducción

Procesamiento en la arquitectura lambda



- ❖ La capa por lotes ejecuta la función sobre el conjunto de datos maestros para precalcular datos intermedios llamados **batch views**.
- ❖ Las **batch views** las carga la **serving layer**, que las indexa para permitir un acceso rápido a esos datos.
- ❖ La **speed layer** compensa la alta latencia de la **batch layer** proporcionando una actualización de latencia más baja utilizando los datos que aún no se han calculado en la **batch view**.

Procesamiento en la arquitectura lambda

— — —



- ❖ **Las queries** se satisfacen procesando datos de serving layer views y de las speed layer views y fusionando el resultado.
- ❖ El eje de la arquitectura es que para **cualquier query** es posible precomputar los datos en la batch layer para acelerar su procesamiento por parte de la serving layer.
- ❖ Las precomputaciones sobre el master dataset toman tiempo, pero se puede tomar la alta latencia de la batch layer como una oportunidad para realizar un análisis profundo de los datos y conectar piezas de datos diversas.

Procesamiento en la arquitectura lambda



— — —

- ❖ Una estrategia ingenua para calcular en la batch layer sería precomputar todas las consultas posibles y guardar el resultado en la cache de la serving layer.
- ❖ Lamentablemente no siempre se puede calcular todo
- ❖ Por otra parte, ya sabemos cómo almacenar el Master Dataset en forma escalable, y con alta tolerancia a fallos y disponibilidad. La pregunta ahora es: ¿cómo procesarlo?

C  mputo Paralelo

Cómputo secuencial

— — —



- ❖ Técnica tradicional en la que se procesan los datos de entrada (input) **en el orden en el que se reciben.**
- ❖ Algoritmos tradicionales que reproducen **linealmente** el problema a resolver.
- ❖ Control **centralizado** de los resultados.
- ❖ El problema se aborda como una **totalidad** y se resuelve de una forma mucho más **natural**

Cómputo secuencial

— — —



Ejemplo. Algoritmo secuencial para **calcular el promedio de edad por ciudad** a partir de los datos del censo.

1. **Recorrer** los datos de cada persona censada.
2. Por cada elemento:
 - a. **extraer** la edad y la ciudad.
 - b. **sumar** uno al contador de personas para esa ciudad.
 - c. **adicionar** la edad a la sumatoria total de edades de esa ciudad.
3. Por cada ciudad, dividir la sumatoria de edades por la cantidad de personas.

Cómputo secuencial

— — —



¿Cómo **escala** un algoritmo de estas características?

- ❖ Si para una población de 1 millón de habitantes tarda 1 minuto, ¿cuánto tardará para una de 100 millones?
- ❖ La alternativa es escalar **verticalmente**

¿De qué formas se puede pensar este algoritmo para **escalarlo horizontalmente**?

Cómputo paralelo

— — —



La técnica alternativa es la del **cómputo paralelo**:

1. Múltiples procesos trabajando en **simultáneo** (sin esperar a que otro proceso termine)
2. Permite aprovechar **múltiples procesadores** (en la misma computadora o en forma distribuida)
 - a. Posibilidad de **escalar horizontalmente**: incorporar más hardware para acelerar una tarea (en lugar de sólo poder escalar verticalmente)
3. Múltiples **partes del input** son procesadas al mismo tiempo.
4. Implica atacar el problema como **diversos subproblemas** equivalentes.

Cómputo paralelo

— — —



Calcular el promedio de edad por ciudad a partir de los datos del censo (revisitado)

1. **Dividir** el input (el censo total) en **n** partes iguales.
2. **Repartir** esas particiones en **n** nodos de cómputo.
3. En cada nodo, recorrer las personas censadas:
 - **extraer** la edad y la ciudad.
 - **sumar** uno al contador de personas para esa ciudad.
 - **adicionar** la edad a la sumatoria total de edades de esa ciudad.

Cómputo paralelo

— — —



Calcular el promedio de edad por ciudad a partir de los datos del censo (revisitado)

4. **Dividir** las **ciudades** (output de los procesos anteriores) en **m** partes iguales
5. **Repartir** esas particiones en **m** nodos de cómputo
6. En cada nodo, recorrer todos los contadores de cada ciudad y calcular el promedio.

Paradigma MapReduce

MapReduce: un paradigma para computar Big Data

— — —



- ❖ **Map Reduce** es un paradigma de computación distribuida del que Google fue pionero y que proporciona primitivas para procesamiento batch de manera escalable y tolerante a fallas.
- ❖ Con Map Reduce se escriben los cálculos en términos de **funciones map y reduce**, funciones que manipulan pares clave-valor.

MapReduce: un paradigma para computar Big Data



- ❖ Estas primitivas son lo suficientemente expresivas como para implementar casi cualquier función, y los **frameworks MapReduce** ejecutan esas funciones sobre el master dataset de una manera distribuida y robusta.
- ❖ Tales propiedades hacen que MapReduce sea un paradigma excelente para el procesamiento necesario en la batch layer, y también un nivel bajo de abstracción donde el procesamiento puede suponer una gran cantidad de trabajo.

MapReduce

— — —



- ❖ La idea en MapReduce es que en cada **etapa** los datos de entrada puedan ser divididos en fragmentos de menor tamaño, a ser ejecutados en distintos nodos de cómputo. La salida producida por estos nodos será luego integrada para generar una salida.

MapReduce

— — —



- ❖ Técnica encadenable

- El resultado de un **reduce** puede ser input de un nuevo **map**

- ❖ Permite escalar horizontalmente tanto como potenciales **claves** (del map) hay en el problema.

- ❖ Implica un **lenguaje común** que permite abordar la resolución por diferentes personas.

- ❖ Se cuenta con **plataformas** que brindan soporte para este patrón y aportan las optimizaciones logradas en la industria (Hadoop, Spark).

MapReduce

— — —



Se estructura en etapas, todas distribuibles **salvo la última**:

- ❖ **Split:** dividir el input en partes de peso equivalente
- ❖ **Map:** estructurar el input de cada parte del split en términos de mapa (clave:valor) en forma paralela
 - *Cada clave implica una subparte del problema que puede computarse independientemente*
- ❖ **Shuffle/Sort:** Repartir de forma equitativa las claves entre procesadores
- ❖ **Reduce:** En cada procesador, resolver la subparte del problema propia de cada clave asignada.
- ❖ **Merge:** juntar los subproductos en un resultado final

MapReduce: etapas “map” y “reduce”



- — —
- ❖ En el **map** se reciben datos de la forma (k, v) (es decir, pares clave-valor) y los *mapea* a cada uno a una lista de pares $[(k_1, v_1), (k_2, v_2)\dots]$ que se pueden calcular como función del par original (k, v) .
 - ❖ En el **reduce** se recibe un par de la forma $(k, [v_1, v_2, \dots, v_n])$ (es decir, una clave y un conjunto de datos asociado a esa clave) y se los resume en un conjunto de valores $[w_1, w_2, \dots, w_m]$ (en general, es un único valor).

MapReduce

— — —



- ❖ El objetivo de dividir nuestro trabajo en secuencias de *maps* y *reduces* es que, de esta forma, sólo nos preocupamos por **pensar el problema de alto nivel**, y no en cómo se resuelven los detalles técnicos del paralelismo y la distribución.
- ❖ Lo único que necesitamos explicarle a MapReduce es qué es lo que queremos computar, y cómo se calcula.

MapReduce

— — —



❖ Data locality

- Los datos de cada partición tienen que estar lo más cerca del nodo posible para reducir *overhead* en la lectura
- ❖ Hay que evitar los problemas de **shuffling** equitativo
- ❖ El **merge** de resultados tiene que ser una tarea lineal sobre un orden muy inferior al del input
 - No puede ser paralelizado

Ejemplo: word count

Word count

— — —

- ❖ El ejemplo canónico de MapReduce es el **recuento de palabras**.
- ❖ Word count toma un dataset de texto y determina la cantidad de veces que apareció cada palabra en el texto.
- ❖ La **función map** en MapReduce se ejecuta una vez por línea de texto y emite una cantidad de pares **clave-valor**
- ❖ Emite un par clave-valor para cada palabra del texto, estableciendo la **clave** para la **palabra** y el **valor** en **1**.
- ❖ MapReduce luego organiza la salida de las funciones map para que todos **los valores con la misma clave** estén agrupados.

Word count

— — —

- ❖ Luego, la **función reduce** toma la lista completa de valores que comparten la misma clave y emite nuevos pares clave-valor como resultado final.
- ❖ La entrada es una lista de un valor para cada palabra y el reduce simplemente suma los valores para calcular la cuenta de esa palabra.
- ❖ Muchas cosas suceden “bajo el capó” para ejecutar un programa como el recuento de palabras en un cluster, pero el framework MapReduce maneja la mayoría de los detalles.
- ❖ La intención es focalizarse en **qué** debe calcularse sin preocuparse por los detalles de **cómo** se calcula.

Ejemplo: word count

— — —

No sé qué es lo que
pasó.
Qué te parece todo
esto?
Nos vemos mañana?
Ya me fui

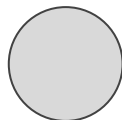
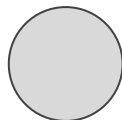
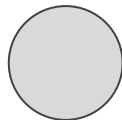
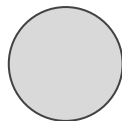
No sé qué es lo que

pasó ¿Qué te parece todo

esto? Nos vemos mañana? Ya

me fui

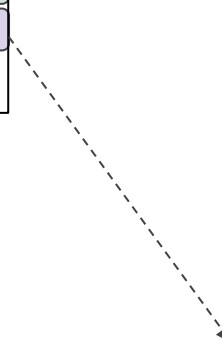
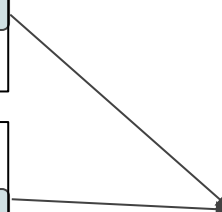
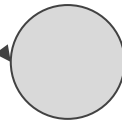
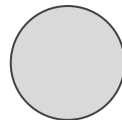
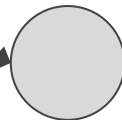
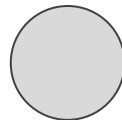
map



no		1
sé		1
qué		2
...		

pasó		1
que		1
te		1
...		

reduce



Ejemplo: word count – pseudo código “map”

— — —

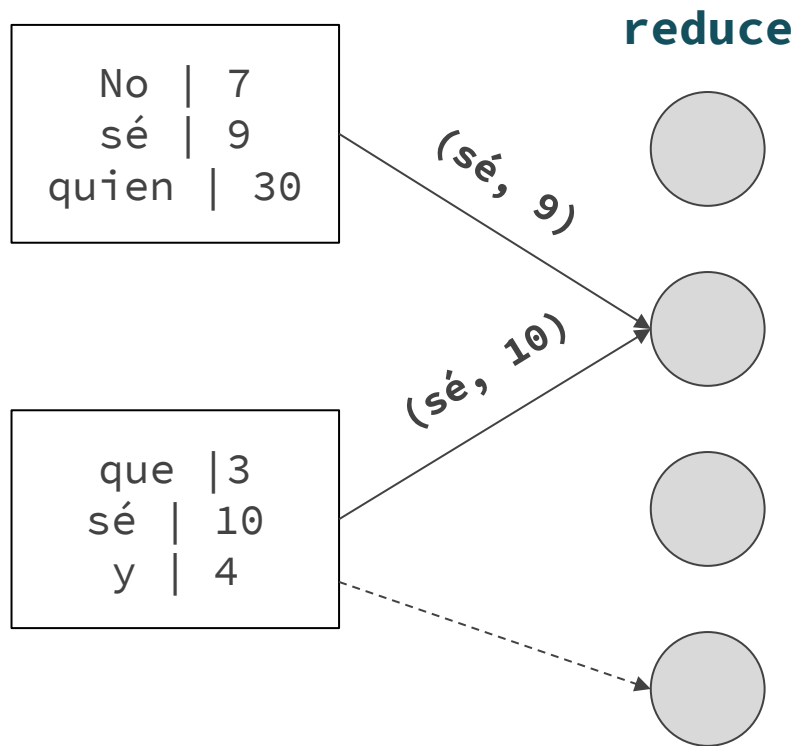
```
def map(k, v):  
    d = dict ()  
    for word in k:  
        if (word in d.keys ()):   
            d[word] = d[word] + 1  
        else:  
            d[word] = 1  
    return d.items ()
```

‘No sé qué es lo que’ | 1

Ejemplo: word count - Proceso de reshuffling

Previo a la función “reduce”, el sistema hace un **reshuffling** que asegura que los pares correspondientes a una misma palabra vayan siempre al mismo nodo de reduce. Para esto se utiliza alguna función de hashing.

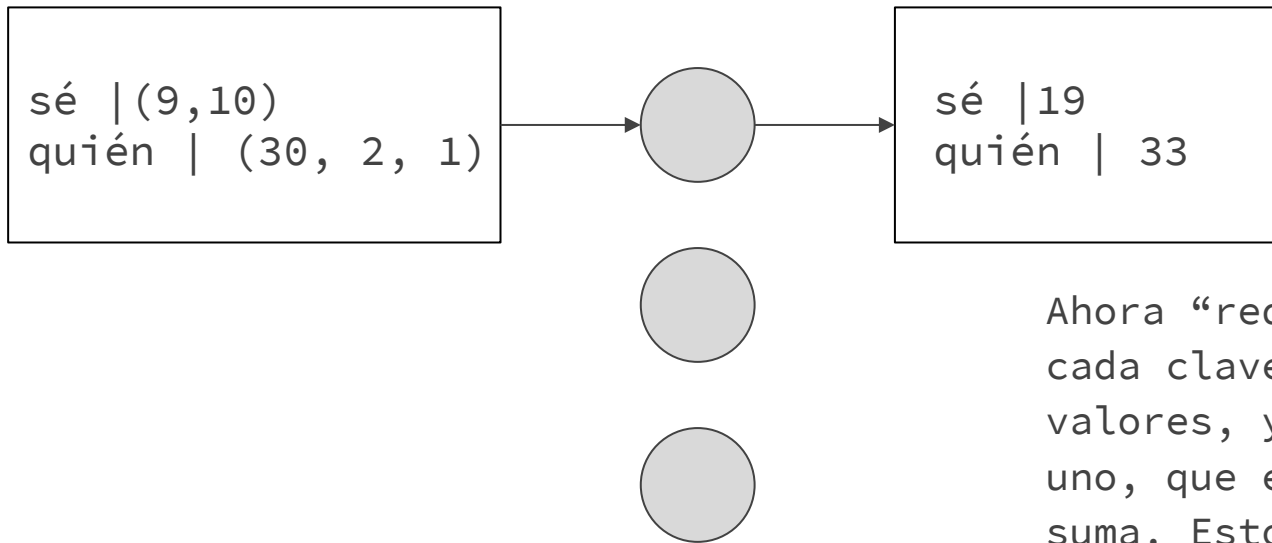
Este reshuffling **NO** tenemos que programarlo nosotros.



Ejemplo: word count - Función “reduce”

— — —

reduce

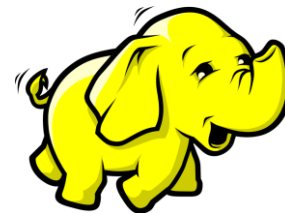


Ahora “reduce” recibirá para cada clave una lista de valores, y los resumirá en uno, que en este caso es la suma. Esto lo programaremos en una función `reduce()`.

Frameworks de cómputo distribuido

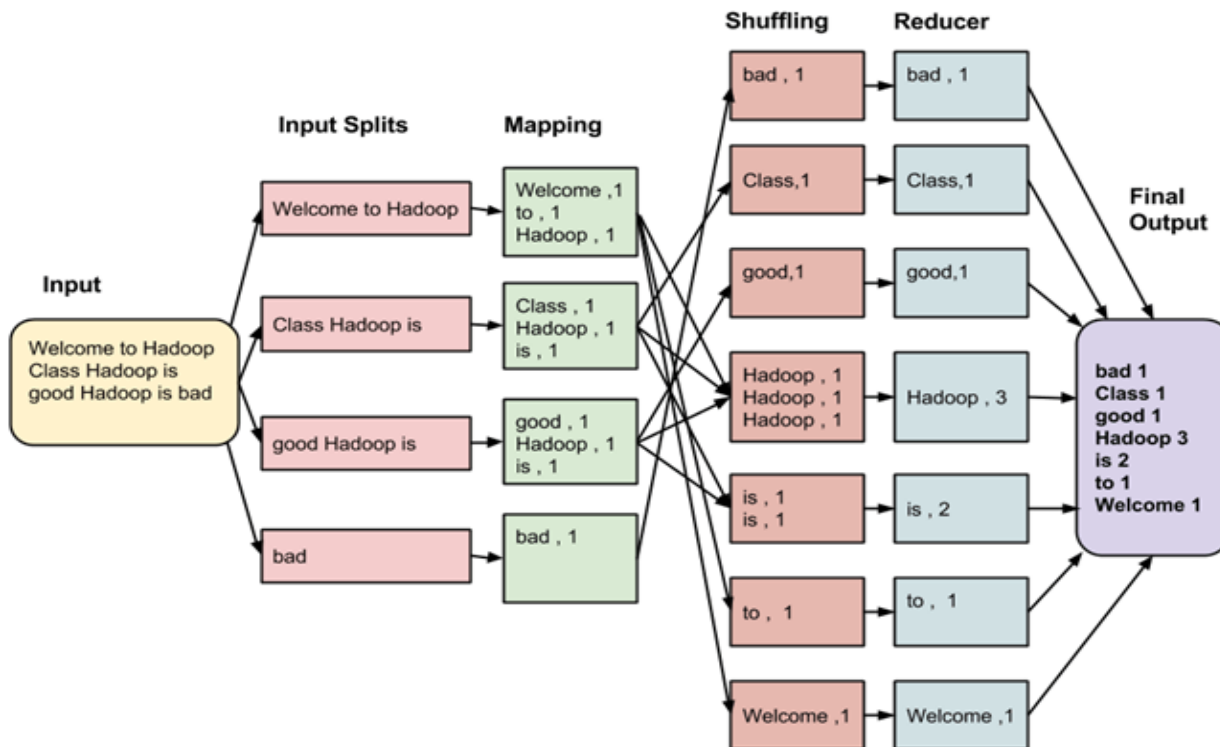
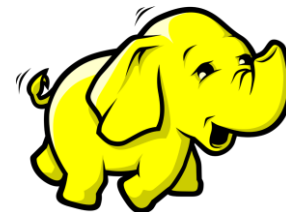
Hadoop MapReduce

— — —

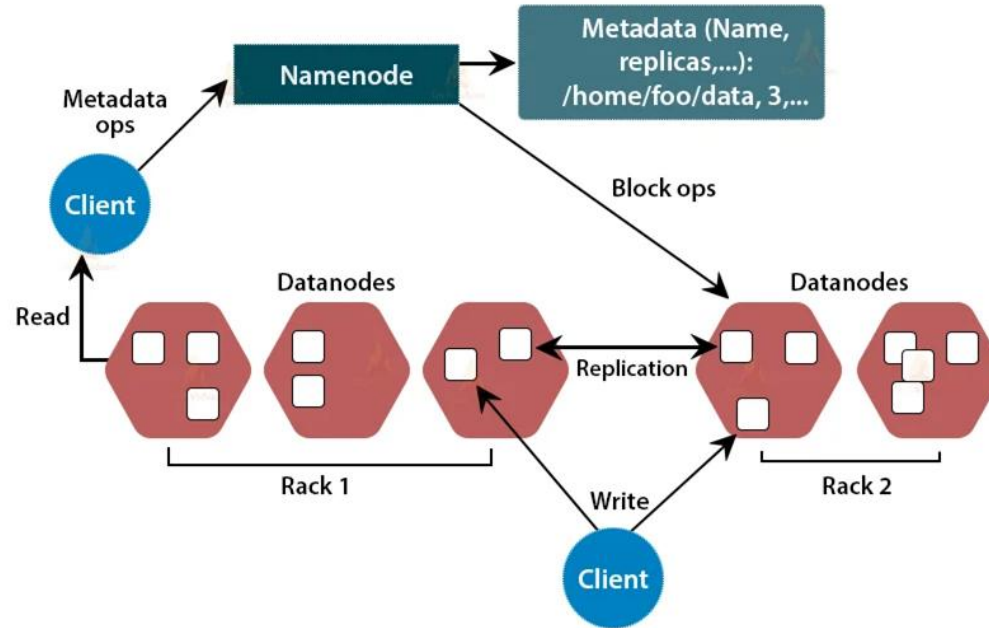


- ❖ Framework open source de **cómputo distribuido**
- ❖ Basado en los fundamentos de **MapReduce** y el **Google Filesystem**
- ❖ Manejo de **recursos** y **scheduling** (**YARN**)
- ❖ **Filesystem propio** orientado a distribución y data locality (**HDFS**)

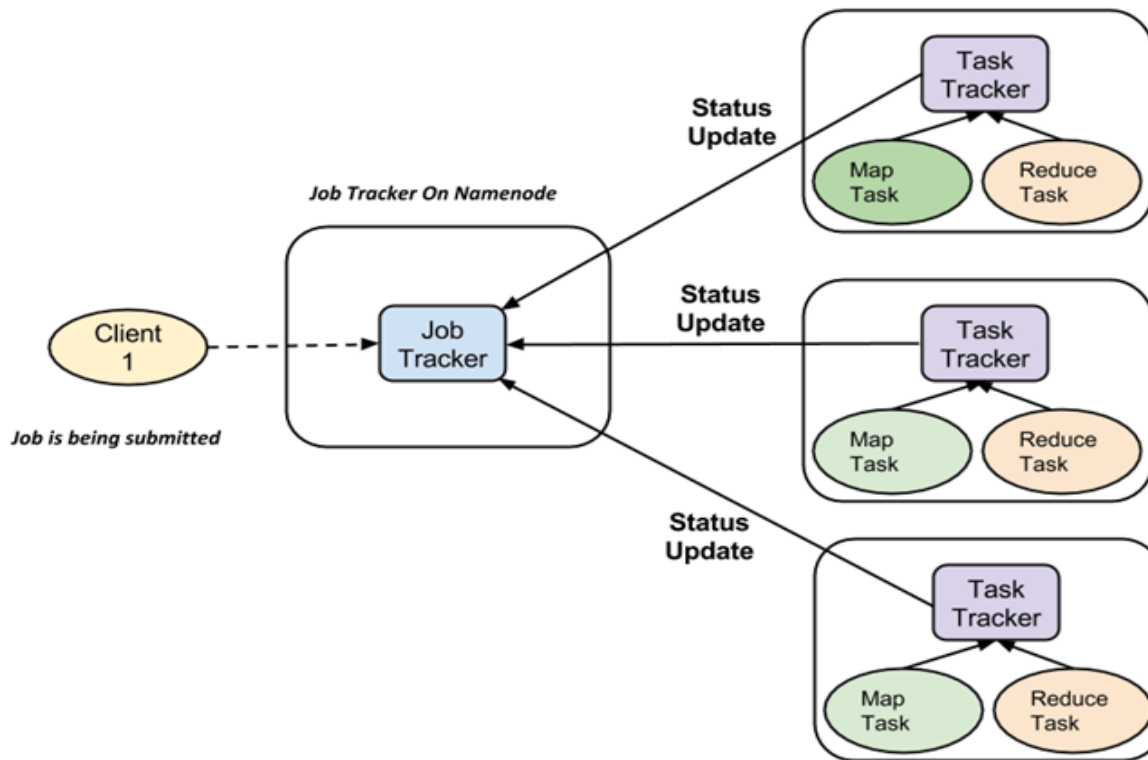
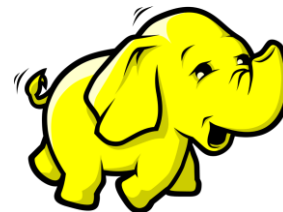
Hadoop MapReduce

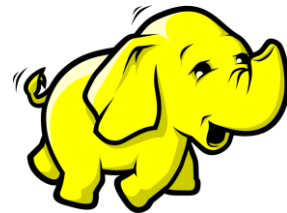


File systems distribuidos - HDFS



Hadoop MapReduce





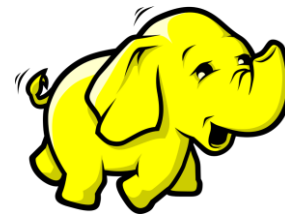
Hadoop MapReduce

- ❖ Un job se divide en varias tareas que luego se ejecutan en varios nodos de datos de un clúster.
- ❖ Es responsabilidad del **job tracker** coordinar la actividad programando tareas para que se ejecuten en diferentes nodos de datos
- ❖ La ejecución de una tarea individual la supervisa el **task tracker**, que reside en cada nodo de datos que ejecuta parte del job.
- ❖ La responsabilidad del task tracker es enviar el informe de progreso al job tracker.
- ❖ Además, el task tracker envía periódicamente heartbeats al job tracker para notificarle el estado actual del sistema.
- ❖ Por lo tanto, el job tracker realiza un seguimiento del progreso general de cada trabajo. En caso de que falle la tarea, el job tracker puede reprogramarla en un task tracker diferente.

<https://hadoop.apache.org/docs/stable/>

Hadoop MapReduce

— — —



Al lanzar un job de MapReduce en Hadoop se dan los siguientes pasos:

1. Splitting / Mapping

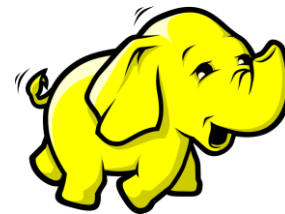
- Se dispara una **Map Task** por cada **bloque** de los archivos a procesar
- Cada task se envía al **Datanode** con el **bloque** elegido o, en su defecto, a otro Datanode en el mismo rack (esto lo determina el **YARN**).
- Cuando la **Map Task** termina, escribe sus resultados en el **disco local** (no en HDFS).

2. Partitioning

- A partir de la configuración del job se define cuántos **Reducers** habrá
- El **Partitioner** determina (a través de una función) a qué reducer corresponde cada **key**.
- Estos resultados también se escriben en el **disco local**

Hadoop MapReduce

— — —



3. Shuffling / Sorting

- Las **Reduce Tasks**, corriendo en distintos **Datanodes** (determinados por el YARN) se traen las **particiones** que les corresponden de los distintos nodos con **Map Tasks**
- Una vez que tienen todas las particiones, **ordenan las keys**

4. Reduce

- Las keys ordenadas alimentan la función de **Reduce**
- El resultado final de las **Reduce Tasks** sí se almacena en **HDFS**

Cierre



Bibliografía

📖 **“Big data: Principles and best practices of scalable realtime data systems”**, N. Marz, J. Warren, 1st edition, 2015.

📌 Capítulo 6: Batch layer

📖 Paper original de MapReduce: **“MapReduce: Simplified Data Processing on Large Clusters”**, J. Dean, S. Ghemawat.
<https://static.googleusercontent.com/media/research.google.com/en//archive/mapreduce-osdi04.pdf>

📖 Cómo implementar correctamente un MapReduce, video de Jeffrey Ullman: <https://www.youtube.com/watch?v=mpCJRq1XruA>